Topics Covered:

- Exponential Coordinates for Rotations and Transformations
- Manipulator Analysis (Joint Space and Workspace)
- Forward Kinematics
- Inverse Kinematics

Additional Reading:

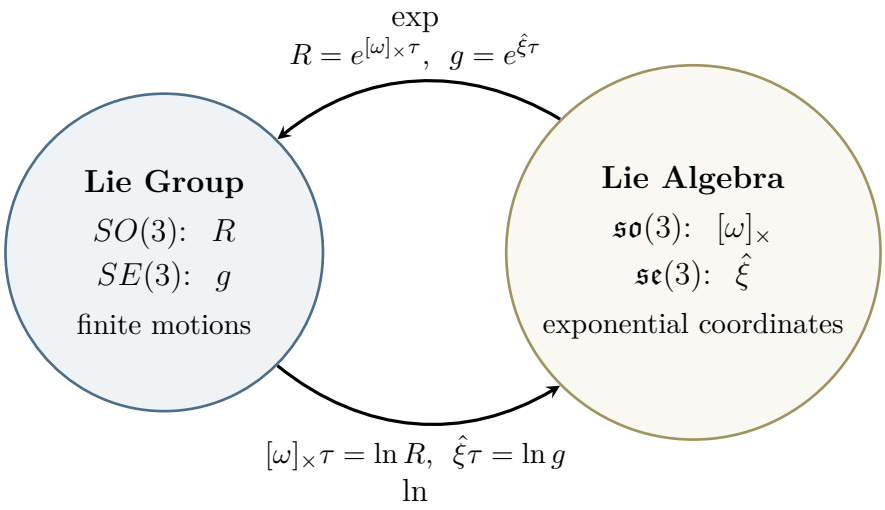
- MLS Chapters 2-3
- LP Chapters 3-6

Note: You are also responsible for the material covered in the Midterm 1 Review (Lecture 9). These notes focus on the material from Lecture 7 to Lecture 13. This review is not necessarily exhaustive, but it covers the main topics that you should focus on for the exam.

Note 2: The best practice problems to study from are your homeworks, the Midterm 2 practice problems, and the example problems from lecture.

Exponential Coordinates

The exponential map and the logarithm let us move back and forth between the Lie group (finite rotations and rigid-body motions) and its Lie algebra (axis-angle coordinates, angular velocities, and twists):



Exponential Coordinates for Rotations

A net rotation can be described with either the unit axis and angle ($[\hat{\omega}]_\times \theta$), or the angular velocity and duration $[\omega]_\times \tau$, using the exponential coordinates:

$$R = \exp([\hat{\omega}]_\times \theta) = \exp([\omega]_\times \tau), \qquad \text{i.e.,} \quad \hat{\omega} = \frac{\omega}{\|\omega\|}, \quad \theta = \|\omega\|\tau.$$

Here, $[\cdot]_\times$ is the skew-symmetric matrix that represents the Lie-algebra element in $\mathfrak{so}(3)$. For any $a \in \mathbb{R}^3$,

$$[a]_\times = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}$$

If we instead wanted to solve for the axis and angle $(\hat{\omega}, \theta)$ given a rotation matrix R, we can use the inverse of the exponential map which is called the *Logarithm*:

$$(\hat{\omega}, \theta) = \ln(R)$$

For rotations, there are relatively simple closed-form solutions to the exponential map and

logarithm. The exponential map is given by the Rodrigues' formula:

$$\exp([\hat{\omega}]_\times \theta) = I + \sin(\theta)[\hat{\omega}]_\times + (1 - \cos(\theta))[\hat{\omega}]_\times^2 \qquad \text{(Unit Axis Formula)}$$

$$\exp([\omega]_\times \tau) = I + \frac{[\omega]_\times}{\|\omega\|}\sin(\|\omega\|\tau) + \frac{[\omega]_\times^2}{\|\omega\|^2}(1 - \cos(\|\omega\|\tau)) \qquad \text{(General Formula)}$$

The logarithm is given by the following formulas:

$$\theta = \cos^{-1}\left(\frac{\mathrm{Tr}(R) - 1}{2}\right) \qquad (\mathrm{Tr} = r_{11} + r_{22} + r_{33})$$

$$\hat{\omega} = \frac{1}{2\sin(\theta)}\begin{bmatrix} r_{32} - r_{23} \\ r_{13} - r_{31} \\ r_{21} - r_{12} \end{bmatrix}$$

If an angular velocity is desired instead (for example, to rotate from one orientation to another in a time τ), it follows from the equivalence: $\omega = \hat{\omega}\theta/\tau$.

Exponential Coordinates for Transformations

As with rotations, we can describe the configuration of a robot manipulator using the exponential coordinates:

$$g(\xi, \tau) = \exp(\hat{\xi}\tau)$$

where here, $\xi \in \mathbb{R}^6$ is a vector that describes the twist (a generalized velocity) and $\tau \in \mathbb{R}$ is the duration. (Note that the hat on $\hat{\xi}$ means the 4×4 matrix form of the twist, not a unit vector.) The twist represents the Lie-algebra element $\mathfrak{se}(3)$ of the SE(3) group is given by:

$$\hat{\xi} = \begin{bmatrix} [\omega]_\times & v \\ 0 & 0 \end{bmatrix}, \quad \xi = \begin{bmatrix} v \\ \omega \end{bmatrix}$$

Also, as with rotations, we can undo this operation using the Logarithm:

$$(\xi, \tau) = \ln(g)$$

Closed form solutions for these expressions are available, albeit much more complicated than the rotation case. These equations are used in the following example.

Closed-Form Exponential and Logarithm

Throughout, the twist $\xi = (v, \omega)$ is applied for a duration τ, $\theta = \|\omega\|\tau$ is the net rotation angle, and $g = (d, R)$ with $R = [r_{ij}]$.

SE(2) (with $v \in \mathbb{R}^2$, $\omega \in \mathbb{R}$, and $\mathbb{J} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$):

$$\exp(\hat{\xi}\tau) = \begin{cases} \left[\begin{array}{c|c} R(\omega\tau) & \frac{1}{\omega}\left(I - R(\omega\tau)\right)\mathbb{J}v \\ \hline 0 & 1 \end{array}\right] & \omega \neq 0 \\ \left[\begin{array}{c|c} I & v\tau \\ \hline 0 & 1 \end{array}\right] & \omega = 0 \end{cases}$$

$$\ln(g): \quad \omega = \frac{1}{\tau}\text{atan2}(r_{21}, r_{11}), \qquad v = \begin{cases} -\omega\mathbb{J}\left(I - R\right)^{-1} d & \omega \neq 0 \\ \frac{1}{\tau} d & \omega = 0 \end{cases}$$

SO(3):

$$\exp([\omega]_\times \tau) = \begin{cases} I + \frac{[\omega]_\times}{\|\omega\|}\sin(\theta) + \frac{[\omega]_\times^2}{\|\omega\|^2}\left(1 - \cos(\theta)\right) & \omega \neq 0 \\ I & \omega = 0 \end{cases}$$

For the logarithm, $\theta = \cos^{-1}\left(\frac{\text{Tr}(R) - 1}{2}\right) \in [0, \pi]$ and $\omega = \hat{\omega}\theta/\tau$, where the unit axis is:

$$\hat{\omega} = \begin{cases} \text{undefined (any axis works, since } \omega = 0) & \theta = 0 \quad (R = I) \\ \frac{1}{2\sin(\theta)}\begin{bmatrix} r_{32} - r_{23} \\ r_{13} - r_{31} \\ r_{21} - r_{12} \end{bmatrix} & 0 < \theta < \pi \\ \pm\frac{c_k}{\|c_k\|}, \quad c_k = \text{any nonzero column of } R + I & \theta = \pi \end{cases}$$

At $\theta = \pi$, R is symmetric, so the general axis formula becomes $0/0$. Instead, Rodrigues' formula gives $R + I = 2\hat{\omega}\hat{\omega}^T$, so every nonzero column of $R + I$ is parallel to $\hat{\omega}$. Both signs are valid, since rotating by π about $\hat{\omega}$ or $-\hat{\omega}$ gives the same R.

SE(3):

$$\exp(\hat{\xi}\tau) = \begin{cases} \left[\begin{array}{c|c} \exp([\omega]_\times \tau) & (I - \exp([\omega]_\times \tau))\dfrac{[\omega]_\times v}{\|\omega\|^2} + \dfrac{\omega\omega^T}{\|\omega\|^2} v\tau \\ \hline 0 & 1 \end{array}\right] & \omega \neq 0 \\ \left[\begin{array}{c|c} I & v\tau \\ \hline 0 & 1 \end{array}\right] & \omega = 0 \end{cases}$$

$$\ln(g): \quad \omega = \text{from the SO(3) logarithm of } R$$

$$v = \begin{cases} \|\omega\|^2 \left((I - R)[\omega]_\times + \tau\omega\omega^T\right)^{-1} d & \omega \neq 0 \\ \dfrac{1}{\tau} d & \omega = 0 \end{cases}$$

The SE(3) logarithm inherits the SO(3) special cases. At $\theta = \pi$, the matrix being inverted is still invertible, so either choice of $\pm\hat{\omega}$ gives a valid twist.

Example Use of Exponential Coordinates

An important use of these expressions though is the ability to describe the configuration of a robot manipulator between two different configurations. For example, consider the following example.

Assume you have object that undergoes a transformation from g_i to g_f where,

$$g_i = \left[\begin{array}{ccc|c} \frac{1}{3} & \frac{1}{\sqrt{2}} & \frac{\sqrt{7}}{3\sqrt{2}} & 1 \\ \frac{1}{3} & -\frac{1}{\sqrt{2}} & \frac{\sqrt{7}}{3\sqrt{2}} & 3 \\ \frac{\sqrt{7}}{3} & 0 & -\frac{2}{3\sqrt{2}} & 2 \\ \hline 0 & & & 1 \end{array}\right] \quad \text{and} \quad g_f = \left[\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & \frac{4}{5} & -\frac{3}{5} & 0 \\ 0 & \frac{3}{5} & \frac{4}{5} & -1 \\ \hline 0 & & & 1 \end{array}\right].$$

The Lie algebra element ξ with time τ can be computed using the Logarithm:

$$(\xi, \tau) = \ln\left(g_i^{-1} g_f\right)$$

which will take the object from g_i to g_f according to the product, $g_f = g_i \exp(\hat{\xi}\tau)$.

Then, the exponential can be used to average the two configurations in a natural way. Since $\exp(\hat{\xi}t)$ for $t \in [0, \tau]$ defines a trajectory from the identity element to $g = \exp(\hat{\xi}\tau)$, we can flow along the path to get a Lie group element between e and g. For example, we can solve for the half-way point between e and g, by solving for the exponential when $t = \tau/2$.

Explicitly, the first step of this process is to compute the tranformation from g_i to g_f:

$$\begin{aligned} g_{if} &= g_i^{-1} g_f \\ &= \left[\begin{array}{c|c} R_i^\top & -R_i^\top d_i \\ \hline 0 & 1 \end{array}\right] \left[\begin{array}{c|c} R_f & d_f \\ \hline 0 & 1 \end{array}\right] \\ &= \begin{bmatrix} 0.3333 & 0.7958 & 0.5055 & -3.6458 \\ 0.7071 & -0.5657 & 0.4243 & 2.1213 \\ 0.6236 & 0.2160 & -0.7513 & -0.4566 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix} \end{aligned}$$

Then, we can compute the Lie-algebra element for the transformation g_{if} as (and choosing $\tau = 1$):

$$\begin{aligned} \|\omega\| &= \frac{1}{\tau}\left(\cos^{-1}\left(\frac{\text{Trace}(R)-1}{2}\right)\right) \\ &= \cos^{-1}\left(\frac{\text{Trace}(R)-1)}{2}\right) \\ &= 3.0136 \\ [\omega]_\times &= \frac{\|\omega\|}{2\sin(\|\omega\|\tau)}(R - R^\top) \\ &= \begin{bmatrix} 0 & 1.0472 & -1.3939 \\ -1.0472 & 0 & 2.4581 \\ 1.3939 & -2.4581 & 0 \end{bmatrix} \\ \omega &= \begin{bmatrix} -2.4581 \\ -1.3939 \\ -1.0472 \end{bmatrix} \\ v &= \|\omega\|^2 \left((I-R)[\omega]_\times + \tau\omega\omega^\top\right)^{-1} d \\ &= \begin{bmatrix} -3.3662 \\ -2.0419 \\ 4.4287 \end{bmatrix} \end{aligned}$$

Then, we can compute any point in time $t \in [0, \tau]$ between g_i and g_f (e.g., the halfway point at $t = \tau/2$). Since we took the logarithm of $g_i^{-1} g_f$, the twist ξ above is the **body** twist ξ_b, so the exponential multiplies $g(0) = g_i$ on the **right**:

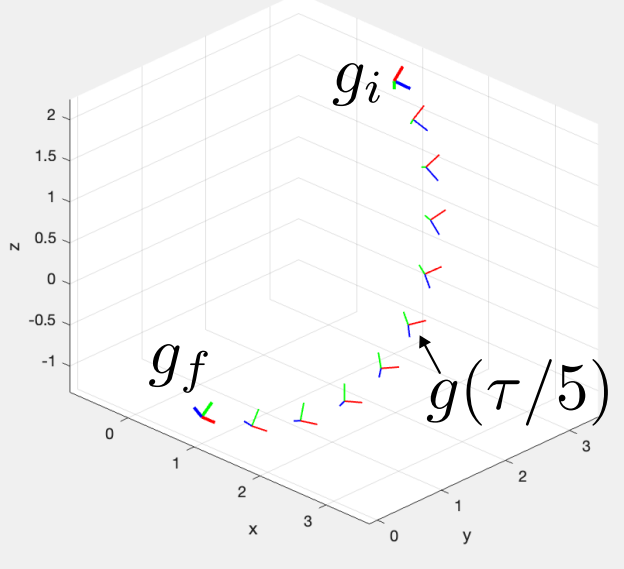
$$g(t) = g(0)\exp(\hat{\xi}_b t), \qquad \xi_b = \ln(g_i^{-1} g_f).$$

If we instead wanted the **spatial** twist ξ_s, we would take the logarithm of $g_f g_i^{-1}$, and the exponential would multiply on the **left**:

$$g(t) = \exp(\hat{\xi}_s t)\, g(0), \qquad \xi_s = \ln(g_f g_i^{-1}) = \text{Ad}_{g_i}\xi_b.$$

The closed-form logarithm is the same in both cases; whether the result is a body or spatial twist depends only on which relative transformation we take the logarithm of. Mixing them up (e.g., $\exp(\hat{\xi}_b t)\, g(0)$) does not reach g_f.

Visually, this gives us the following path:



The Adjoint Operation

The adjoint operation and its inverse are:

$$\mathrm{Ad}_{g_1} g_2 = g_1 g_2 g_1^{-1}, \qquad \mathrm{Ad}_{g_1}^{-1} g_2 = g_1^{-1} g_2 g_1$$

In general, this adjoint operation is used to change the frame of reference. Our classic example is if we wanted to change some transformation that was in the end-effector frame to a transformation of the tool frame. Specifically, assume that we have the transformations g_{we} and $g_{we'}$. We can use these to compute the transformation that the end-effector undergoes by:

$$g_{ee'} = g_{we}^{-1} g_{we'}$$

If we also have the transformation between the end-effector and some tool in the end-effector's grip, g_{et}, then we can use the adjoint to obtain the transformation of the tool:

$$g_{tt'} = g_{et}^{-1} g_{ee'} g_{et} = \mathrm{Ad}_{g_{et}^{-1}} g_{ee'}$$

Always pay attention to the form of your adjoint operation. If you are given a different coordinate frame relationship (such as g_{te}), then your inverse will be in the opposite order:

$$g_{tt'} = g_{te} g_{ee'} (g_{te})^{-1} = \mathrm{Ad}_{g_{te}} g_{ee'}$$

Manipulator Analysis

We typically operate in one of two spaces: Joint Space ($\vec{\theta} \in M$) or Configuration Space ($g \in SE(n)$).

Joint space can be mathematically represented as a higher dimensional Torus of r revolute joints combined with the Cartesian space of p prismatic/helical joints:

$$M = T^r \times \mathbb{R}^p$$
$$r : \# \text{ revolute joints}$$
$$p : \# \text{ prismatic/helical joints}$$

Configuration space is mathematically represented through three definitions of workspace.

Normal workspace (complete workspace) is defined as:

$$W = \{g_e(\vec{\theta}) \in SE(n) \mid \vec{\theta} \in M)\}$$

Reachable workspace is defined as:

$$W_r = \{p_e(\vec{\theta}) \in E(n) \mid \vec{\theta} \in M\}$$

And lastly, dextrous workspace is defined as:

$$W_d = \{p_e(\vec{\theta}) \in E(n) \mid \ \forall R \in SO(n),\ \exists \vec{\theta} \in M \ s.t.\ g_e(\vec{\theta}) = (p_e, R)\}$$

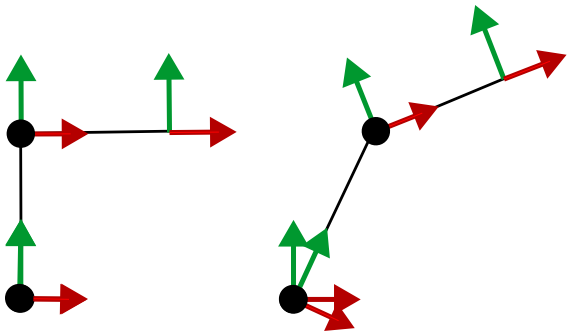
Forward Kinematics

There are two main methods we leveraged for forward kinematics: the product of homogeneous transformation matrices (or sometimes called the product of Lie groups) and the product of exponentials.

The product of homogeneous transformation matrices method uses the formula (for a manipulator with m joints):

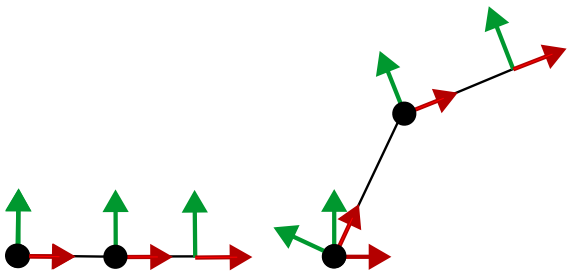
$$g_e(\vec{\theta}) = g_1(\theta_1)g_2(\theta_2)\cdots g_m(\theta_m)g_{m+1}$$

For example, consider the following manipulators, whose forward kinematics are written as a product of homogeneous transformation matrices:



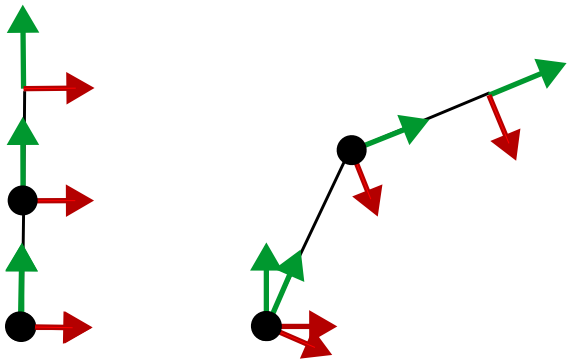
$$g_{WE} = g_{WA}g_{AB}g_{BC}g_{CE} = \begin{bmatrix} R(\theta_1) & \begin{bmatrix}0\\0\end{bmatrix} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R(\theta_2) & \begin{bmatrix}0\\l_1\end{bmatrix} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R(0) & \begin{bmatrix}l_2\\0\end{bmatrix} \\ 0 & 1 \end{bmatrix}$$

Figure 1: Example Manipulator 1: $\vec{\theta} = (\theta_1, \theta_2)$ with lengths l_1 and l_2



$$g_{WE} = g_{WA}g_{AB}g_{BC}g_{CE} = \begin{bmatrix} R(\theta_1) & \begin{bmatrix}0\\0\end{bmatrix} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R(\theta_2) & \begin{bmatrix}l_1\\0\end{bmatrix} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R(0) & \begin{bmatrix}l_2\\0\end{bmatrix} \\ 0 & 1 \end{bmatrix}$$

Figure 2: Example Manipulator 2: $\vec{\theta} = (\theta_1, \theta_2)$ with lengths l_1 and l_2

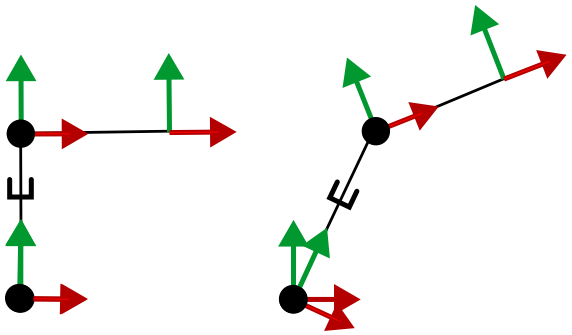


$$g_{WE} = g_{WA}g_{AB}g_{BC}g_{CE} = \begin{bmatrix} R(\theta_1) & \begin{bmatrix}0\\0\end{bmatrix} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R(\theta_2) & \begin{bmatrix}0\\l_1\end{bmatrix} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R(0) & \begin{bmatrix}0\\l_2\end{bmatrix} \\ 0 & 1 \end{bmatrix}$$

Figure 3: Example Manipulator 3: $\vec{\theta} = (\theta_1, \theta_2)$ with lengths l_1 and l_2

For all of these examples, note that you could derive the correct $\vec{d}$ vectors based on just the right-hand images since the frame axes are provided for you. But I've also provided you with the zero configurations on the left for clarity.

If we have robots with prismatic joints, this process is the same, we just have a variable in our $\vec{d}$ vector instead of a constant. For example, consider the following manipulator:



$$g_{WE} = g_{WA}g_{AB}g_{BC}g_{CE} = \begin{bmatrix} R(\theta_1) & \begin{bmatrix}0\\0\end{bmatrix} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R(\theta_3) & \begin{bmatrix}0\\\theta_2\end{bmatrix} \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R(0) & \begin{bmatrix}l_2\\0\end{bmatrix} \\ 0 & 1 \end{bmatrix}$$

Figure 4: Example Manipulator 4: $\vec{\theta} = (\theta_1, \theta_2, \theta_3)$ where θ_2 is now the prismatic joint.

The product of exponentials method uses the formula (for a manipulator with m joints):

$$g_e(\vec{\theta}) = e^{\hat{\xi}_1\theta_1}e^{\hat{\xi}_2\theta_2}\cdots e^{\hat{\xi}_m\theta_m}g_0$$

For the product of exponential method, the twists are obtained using the formulas:

$$\xi_i = \begin{bmatrix} v_i \\ \omega_i \end{bmatrix} = \begin{cases} \begin{bmatrix} -\omega_i \times q_i \\ \omega_i \end{bmatrix} & \text{if revolute} \\ \begin{bmatrix} v_i \\ 0 \end{bmatrix} & \text{if prismatic} \end{cases}$$

In the above formulas for a revolute joint, ω_i is the unit axis of rotation and q_i is any point on the axis of rotation. For the formula for the prismatic joint, v_i is the unit axis of translation. All variables are relative to the spatial (fixed) frame, and are taken at the zero configuration.

The twist is "hatted" by converting the vector into homogeneous form:

$$\xi_i = \begin{bmatrix} v_i \\ \omega_i \end{bmatrix} \underset{\text{hatting}}{\rightarrow} \hat{\xi}_i = \begin{bmatrix} [\omega_i]_\times & v_i \\ 0 & 0 \end{bmatrix}$$

This homogeneous form can be put back into vector form by "unhatting" or "veeing" the matrix:

$$\hat{\xi}_i = \begin{bmatrix} [\omega_i]_\times & v_i \\ 0 & 0 \end{bmatrix} \underset{\text{unhatting}}{\rightarrow} \xi_i = (\hat{\xi}_i)^\vee = \begin{bmatrix} v_i \\ \omega_i \end{bmatrix}$$

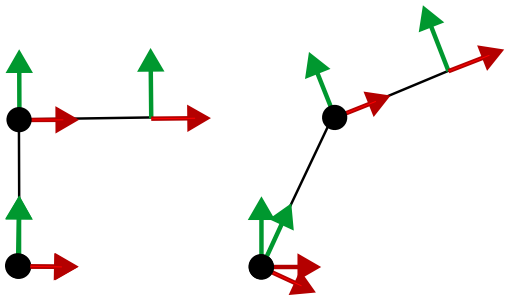
Examples: Product of Exponentials

We can now repeat the example manipulators above using the product of exponentials. To keep the cross products straightforward, we treat them in 3D: $\hat{x}$ points right, $\hat{y}$ points up

the page, and $\hat{z}$ points out of the page. Every revolute joint rotates about $\omega = \hat{z} = (0, 0, 1)$, so for a joint located at q in the zero configuration,

$$\xi = \begin{bmatrix} -\omega \times q \\ \omega \end{bmatrix}, \qquad \text{e.g.,} \quad -\hat{z} \times \begin{bmatrix} 0 \\ l_1 \\ 0 \end{bmatrix} = -\begin{bmatrix} -l_1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} l_1 \\ 0 \\ 0 \end{bmatrix}, \quad -\hat{z} \times \begin{bmatrix} l_1 \\ 0 \\ 0 \end{bmatrix} = -\begin{bmatrix} 0 \\ l_1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ -l_1 \\ 0 \end{bmatrix}.$$

All twists and g_0 are taken at the zero configuration ($\vec{\theta} = 0$), relative to the world frame W, and the end-effector frame starts aligned with W (so g_0 has $R = I$).



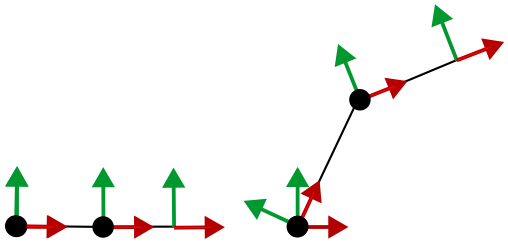
$$\xi_1 = \begin{bmatrix} -\hat{z} \times 0 \\ \hat{z} \end{bmatrix} = (0, 0, 0, 0, 0, 1)^\top$$

$$\xi_2 = \begin{bmatrix} -\hat{z} \times q_2 \\ \hat{z} \end{bmatrix} = (l_1, 0, 0, 0, 0, 1)^\top$$

$$g_0 = \begin{bmatrix} I & \begin{bmatrix} l_2 \\ l_1 \\ 0 \end{bmatrix} \\ 0 & 1 \end{bmatrix}$$

$$g_{WE}(\vec{\theta}) = e^{\hat{\xi}_1 \theta_1} e^{\hat{\xi}_2 \theta_2} g_0$$

Figure 5: Example Manipulator 1 with the product of exponentials: joint 2 is at $q_2 = (0, l_1, 0)$ and the end-effector is at $(l_2, l_1, 0)$ in the zero configuration.



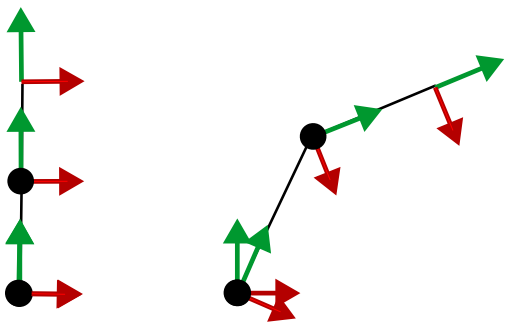
$$\xi_1 = \begin{bmatrix} -\hat{z} \times 0 \\ \hat{z} \end{bmatrix} = (0, 0, 0, 0, 0, 1)^\top$$

$$\xi_2 = \begin{bmatrix} -\hat{z} \times q_2 \\ \hat{z} \end{bmatrix} = (0, -l_1, 0, 0, 0, 1)^\top$$

$$g_0 = \begin{bmatrix} I & \begin{bmatrix} l_1 + l_2 \\ 0 \\ 0 \end{bmatrix} \\ 0 & 1 \end{bmatrix}$$

$$g_{WE}(\vec{\theta}) = e^{\hat{\xi}_1 \theta_1} e^{\hat{\xi}_2 \theta_2} g_0$$

Figure 6: Example Manipulator 2 with the product of exponentials: joint 2 is at $q_2 = (l_1, 0, 0)$ and the end-effector is at $(l_1 + l_2, 0, 0)$ in the zero configuration.



$$\xi_1 = \begin{bmatrix} -\hat{z} \times 0 \\ \hat{z} \end{bmatrix} = (0,0,0,0,0,1)^\top$$

$$\xi_2 = \begin{bmatrix} -\hat{z} \times q_2 \\ \hat{z} \end{bmatrix} = (l_1,0,0,0,0,1)^\top$$

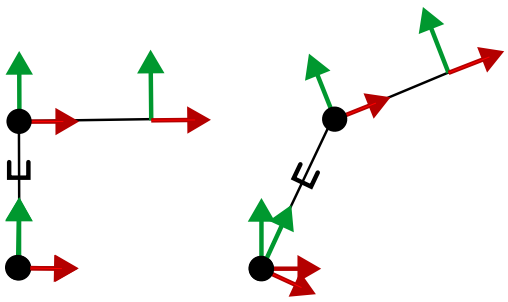
$$g_0 = \begin{bmatrix} I & \begin{bmatrix} 0 \\ l_1 + l_2 \\ 0 \end{bmatrix} \\ 0 & 1 \end{bmatrix}$$

$$g_{WE}(\vec{\theta}) = e^{\hat{\xi}_1 \theta_1} e^{\hat{\xi}_2 \theta_2} g_0$$

Figure 7: Example Manipulator 3 with the product of exponentials: joint 2 is at $q_2 = (0, l_1, 0)$ and the end-effector is at $(0, l_1 + l_2, 0)$ in the zero configuration.

Notice that Manipulators 1 and 3 have the same twists, since their joints are in the same places; only g_0 (the end-effector's zero-configuration pose) changes.

For the prismatic example, the prismatic joint slides along $\hat{y}$, so its twist is $\xi_2 = (v_2, 0)$ with $v_2 = \hat{y} = (0,1,0)$. Following the product of homogeneous transformations above, the offset is $(0, \theta_2)$, so in the zero configuration ($\theta_2 = 0$) joint 3 sits at the origin:



$$\xi_1 = \begin{bmatrix} -\hat{z} \times 0 \\ \hat{z} \end{bmatrix} = (0,0,0,0,0,1)^\top$$

$$\xi_2 = \begin{bmatrix} \hat{y} \\ 0 \end{bmatrix} = (0,1,0,0,0,0)^\top$$

$$\xi_3 = \begin{bmatrix} -\hat{z} \times 0 \\ \hat{z} \end{bmatrix} = (0,0,0,0,0,1)^\top$$

$$g_0 = \begin{bmatrix} I & \begin{bmatrix} l_2 \\ 0 \\ 0 \end{bmatrix} \\ 0 & 1 \end{bmatrix}$$

$$g_{WE}(\vec{\theta}) = e^{\hat{\xi}_1 \theta_1} e^{\hat{\xi}_2 \theta_2} e^{\hat{\xi}_3 \theta_3} g_0$$

Figure 8: Example Manipulator 4 with the product of exponentials: θ_2 is the prismatic joint.

In each case, multiplying out the product of exponentials gives exactly the same $g_{WE}(\vec{\theta})$ as the product of homogeneous transformation matrices (with each planar transformation embedded in 3D as a rotation about $\hat{z}$).

Inverse Kinematics

There are three main methods that we covered for inverse kinematics: the geometric approach, Newton-Raphson (an iterative numerical approach), and an optimization-based approach. We also covered resolved-rate trajectory generation which can be thought of as an approach to "inverse kinematics" (by taking a linear integration step of our desired velocity), but we will cover this later in the course.

Geometric Approach

For simple manipulator arms, we can use the geometric approach to directly solve for joint angles required to achieve a desired end-effector position.

This is accomplished by drawing the triangle formed by the end-effector position and a two-link manipulator arm, and using the law of cosines to solve for the corresponding joint angles. We can then use our standard equations for either the "Lefty" (also known as "Elbow-Up") configuration:

$$\theta_1 = \gamma + \alpha$$
$$\theta_2 = \beta - \pi$$

For the "Righty" (also known as "Elbow-Down") configuration we instead use the equations:

$$\theta_1 = \gamma - \alpha$$
$$\theta_2 = \pi - \beta$$

With α and β solved for using the law of cosines:

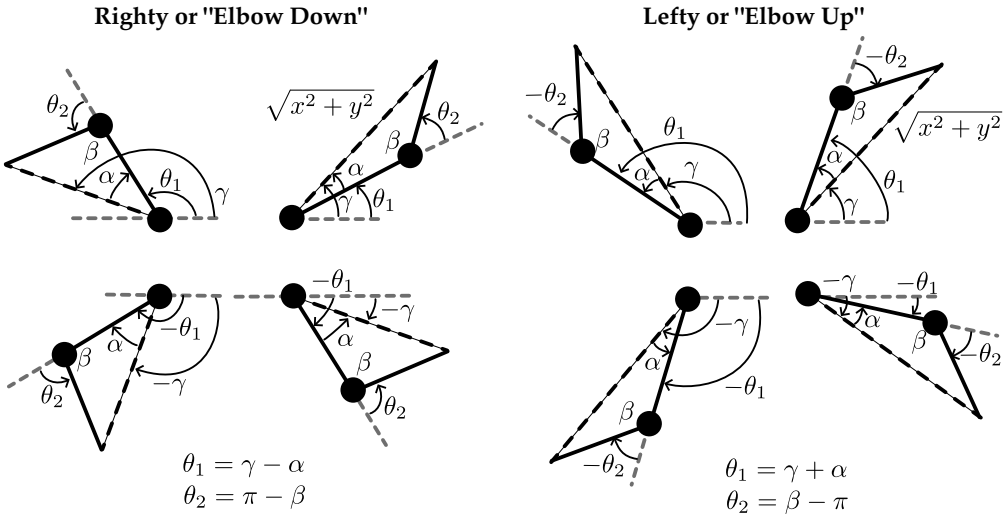
$$\alpha = \cos^{-1}\left(\frac{x^2 + y^2 + l_1^2 - l_2^2}{2l_1\sqrt{x^2 + y^2}}\right)$$
$$\beta = \cos^{-1}\left(\frac{l_1^2 + l_2^2 - x^2 - y^2}{2l_1 l_2}\right)$$

And with γ solved for using the two-argument arctangent function:

$$\gamma = \text{atan2}(y, x)$$

We can ignore $\pm\cos^{-1}$ as long as you pay careful attention to the arguments of arccosine and which solution you utilize in which quadrant.

For clarity, there's each of the labels for the Lefty and Righty configurations when the manipulator is in each of the four quadrants:



Newton-Raphson

The Newton-Raphson method is an iterative root-finding algorithm that can be used for inverse kinematics. The general equation for Newton-Raphson is:

$$\theta_{k+1} = \theta_k + \left(\frac{\partial f}{\partial \theta}(\theta_k)\right)^{-1}(x_d - f(\theta_k)) = \theta_k + J^{-1}(\theta_k)(x_d - f(\theta_k)),$$

with $f(\theta)$ being the forward kinematics mapping of the end-effector position and $J(\theta) = \frac{\partial f}{\partial \theta}$ its Jacobian. When J is not square, the inverse is replaced by the pseudoinverse $J^\dagger$ (see the Manipulator Jacobian lectures). This algorithm is continued until the difference between θ_{k+1} and θ_k is less than a specified tolerance.

Optimization-Based Inverse Kinematics

Lastly, we commonly leverage modern optimization techniques to solve for inverse kinematics. This is done by minimizing the error between the desired end-effector position and the actual end-effector position by solving an optimization problem of the form:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \ \|x_d - f(\theta)\|^2$$
$$\text{subject to } \theta_{\min} \leq \theta \leq \theta_{\max}$$

These optimization problems are commonly solved by numerically approximating the gradient of the cost function and using a gradient-based optimization algorithm such as gradient descent or L-BFGS.